

Machine Learning with TensorFlow

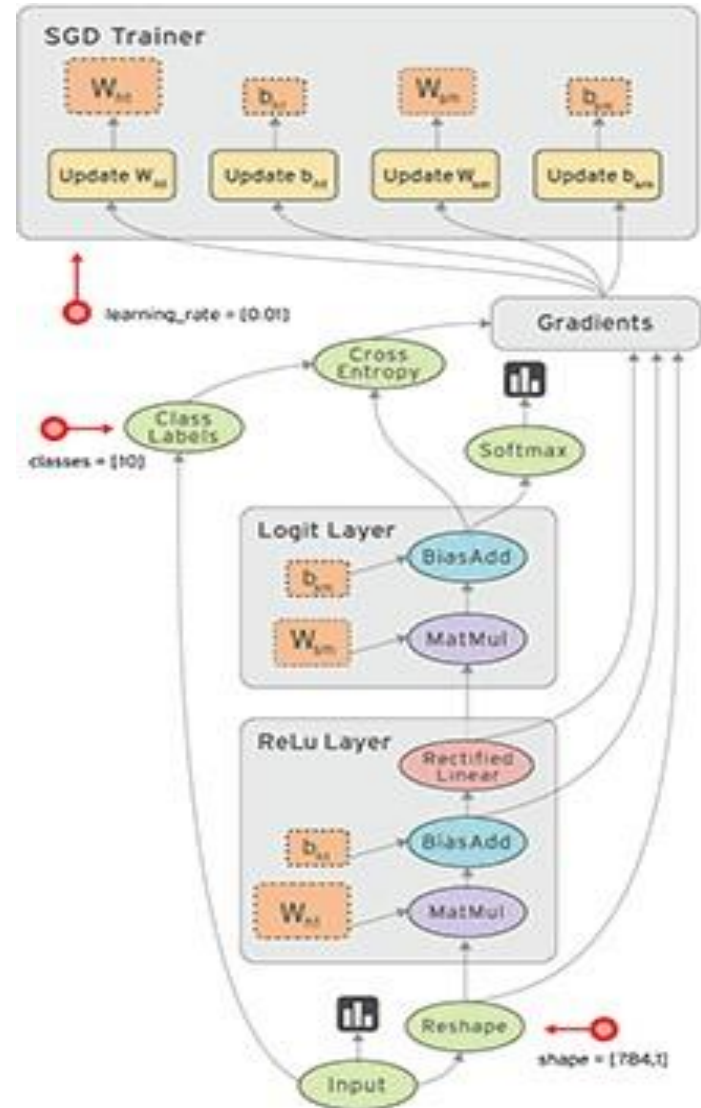
Deep Learning with TensorFlow

MACHINE LEARNING WITH TENSORFLOW

👉 실습

● TensorFlow

- TensorFlow is an open source software library for numerical computation using data flow graphs.
- Data flow graph는 **Node**와 **Edge**로 구성된 방향성 있는 그래프로 표현
- Node는 mathematical operation 데이터의 입출력 작업을 수행
- Edge는 동적 크기의 다차원배열(Tensor)을 Node로 실어 나르는 역할



MACHINE LEARNING WITH TENSORFLOW

👉 실습

- 아래의 코드를 이용하여 Hello World 출력

- Node는 mathematical operation, 데이터의 입출력 작업을 수행.

```
import tensorflow as tf  
  
node = tf.constant("Hello World")  
  
sess = tf.Session()  
  
print(sess.run(node).decode())
```

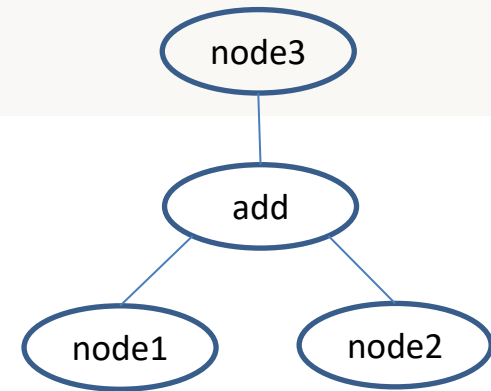
```
Hello World
```

<http://stackoverflow.com/questions/6269765>

MACHINE LEARNING WITH TENSORFLOW

👉 실습

● 간단한 수학 연산 수행



```
import tensorflow as tf

node1 = tf.constant(10, dtype=tf.float32)
node2 = tf.constant(20, dtype=tf.float32)

node3 = tf.add(node1, node2) # node3 = node1 + node2

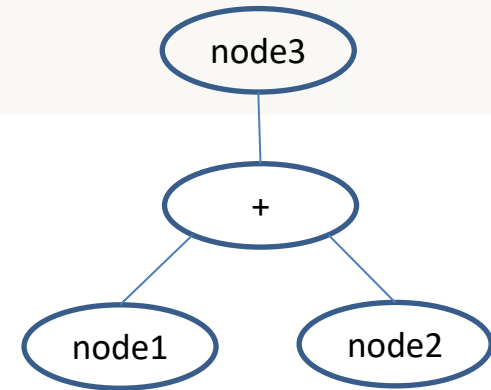
sess = tf.Session()

# print(sess.run([node1, node2]))
print(sess.run(node3))
```

```
[10.0, 20.0]
30.0
```

MACHINE LEARNING WITH TENSORFLOW

👉 실습



- placeholder 사용 그래프를 실행시키는 단계에서 값을 던져 줌

```
# placeholder 사용
# 2개의 수를 입력으로 받아서 덧셈 연산
import tensorflow as tf

node1 = tf.placeholder(dtype=tf.float32)
node2 = tf.placeholder(dtype=tf.float32)

node3 = node1 + node2

sess = tf.Session()

result = sess.run(node3, feed_dict={ node1 : 10, node2 : 20})
# result = sess.run(node3, feed_dict={ node1 : input(), node2 : input()})
# result = sess.run(node3, feed_dict={ node1 : [1,2,3], node2 : [10,20,30]})

print("덧셈연산결과 : {}".format(result))
```

MACHINE LEARNING WITH TENSORFLOW

☞ Supervised Learning Type(1)

● Linear Regression

➤ 7시간 공부하면 0 ~ 100점 중 몇 점을 받을 수 있는가?

x(시간)	y(점수)
10	95
8	80
5	68
2	15
1	5

MACHINE LEARNING WITH TENSORFLOW

☞ Supervised Learning Type(2)

● Logistic Regression (Binary Classification)

➤ 7시간 공부하면 합격인가 혹은 불합격 인가? (2개 중 1개 , 0 or 1)

x(시간)	y(결과)
10	PASS(1)
8	PASS(1)
5	FAIL(0)
2	FAIL(0)
1	FAIL(0)

MACHINE LEARNING WITH TENSORFLOW

☞ Supervised Learning Type(3)

● Multinomial Classification

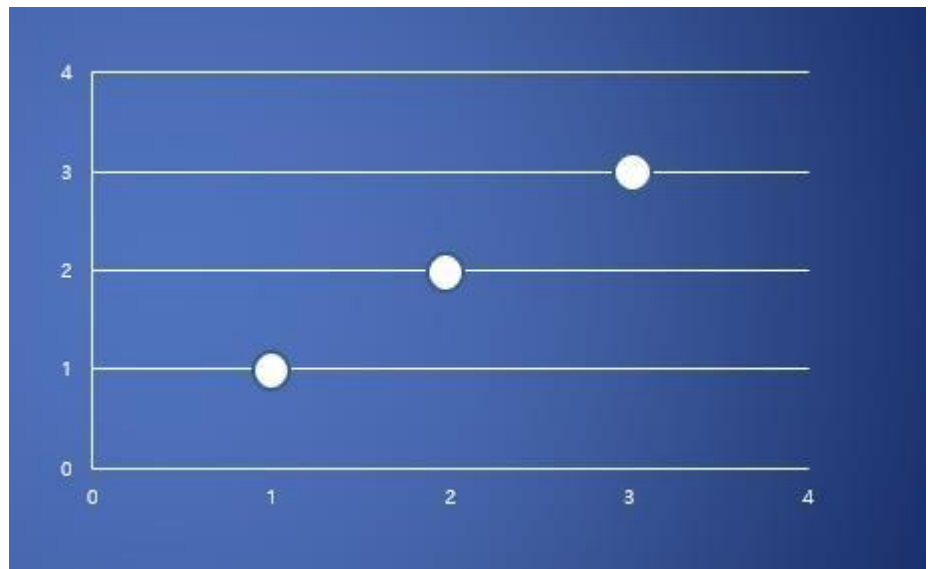
➤ 7시간 공부하면 어떤 grade를 받을 수 있는가? (A,B,C,D,F – N개 중 1개)

x(시간)	y(결과)
10	A
8	B
5	D
2	F
1	F

MACHINE LEARNING WITH TENSORFLOW

👉 Linear Regression – 사용할 training data set

x	y
1	1
2	2
3	3



MACHINE LEARNING WITH TENSORFLOW

☞ Linear Hypothesis

- 많은 데이터와 현상들이 linear한 형태를 가진다.

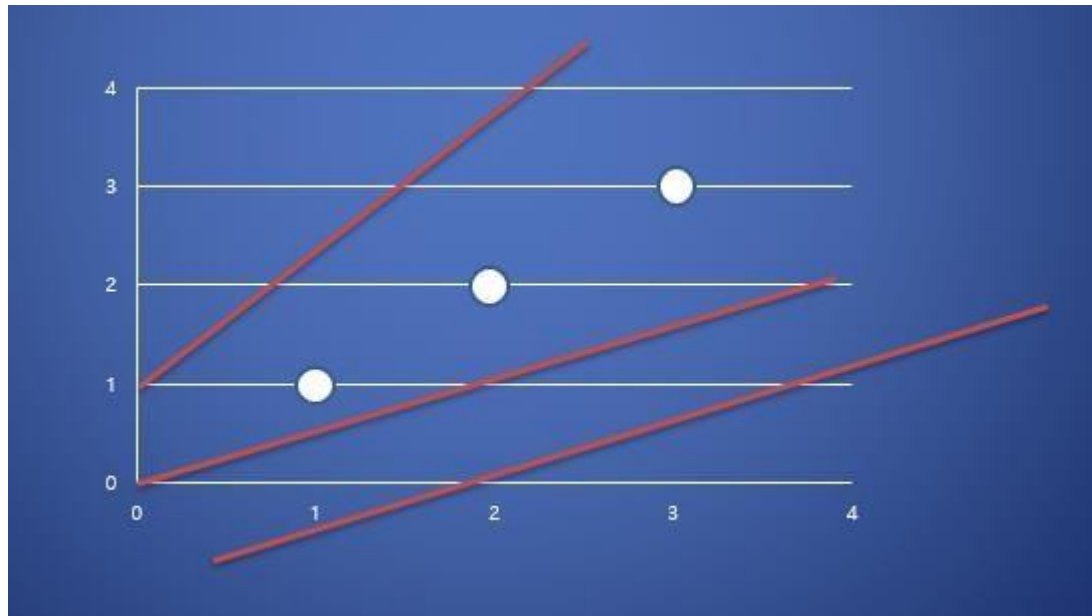
- 일반적으로,
 - 많은 시간 공부를 하면 점수를 더 많이 받고
 - 오랜 시간 일을 하면 더 많은 돈을 벌고
 - 공을 세게 던지면 던질수록 더 멀리 나아가고
 - 배달지역이 멀수록 배달시간이 오래 걸린다.

- 우리 예제도 이러한 linear한 형태라고 가정하고 하나의 가설(Hypothesis)을 세운다.

MACHINE LEARNING WITH TENSORFLOW

☞ Linear Hypothesis

- Linear Hypothesis를 정의하는 것은 training data set에 잘 맞는 linear한 선을 긋는 것으로 생각할 수 있다.
- Hypothesis를 수정해 나가면서 데이터에 가장 적합한 선을 찾는 과정이 바로 학습.



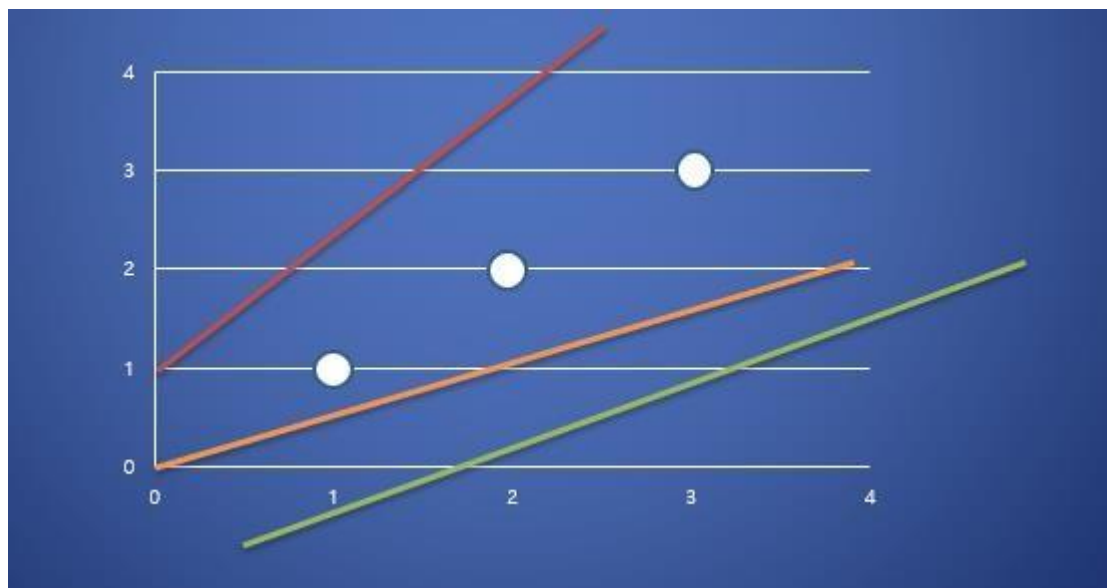
MACHINE LEARNING WITH TENSORFLOW

☞ Linear Hypothesis

- 이러한 선들을 수학적으로 표현하면 다음과 같다.

$$H(x) = Wx + b$$

Hypothesis(가설)

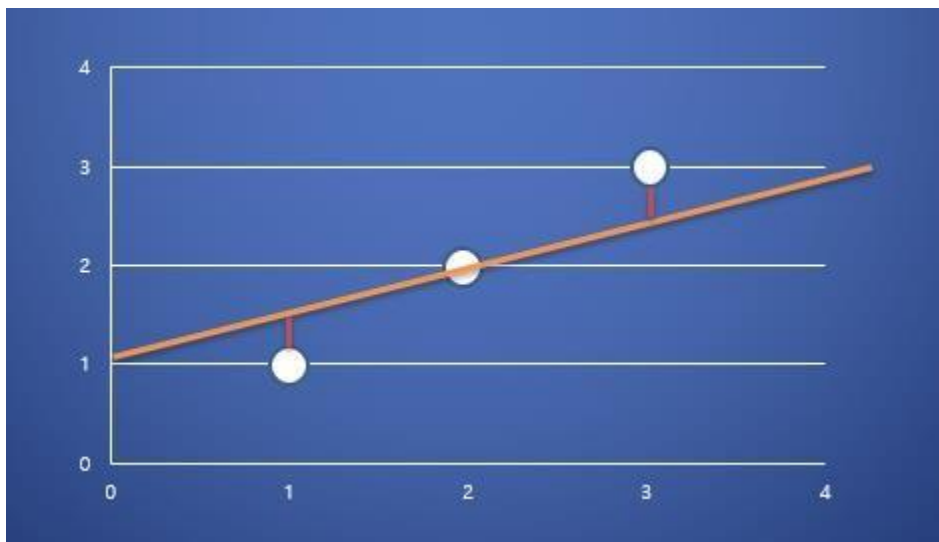


MACHINE LEARNING WITH TENSORFLOW

☞ Linear Hypothesis

- 우리가 원하는 것은 training data set에 가장 적합한 W 와 b 의 값을 찾는 것.
 - 가장 적합한 W 와 b 의 값을 찾으려면 **어떻게** 해야 하는가?
 - Cost(Loss) Function 사용 - 최소제곱법

$$H(x) = Wx + b$$

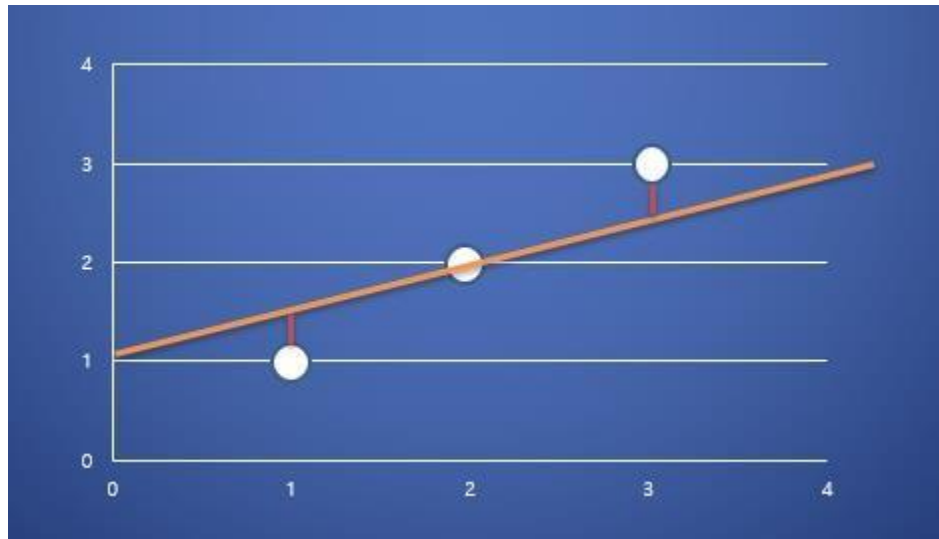


MACHINE LEARNING WITH TENSORFLOW

☞ Cost(Loss) Function

- 가설과 training data간의 값의 차를 계산하는 함수 $H(x) = Wx + b$

$$\frac{(H(x^{(1)}) - y^{(1)})^2 + (H(x^{(2)}) - y^{(2)})^2 + (H(x^{(3)}) - y^{(3)})^2}{3}$$



MACHINE LEARNING WITH TENSORFLOW

☞ Cost(Loss) Function

- 가설과 training data간의 값의 차를 계산하는 함수

$$\frac{(H(x^{(1)}) - y^{(1)})^2 + (H(x^{(2)}) - y^{(2)})^2 + (H(x^{(3)}) - y^{(3)})^2}{3}$$



$$cost(W, b) = \frac{1}{n} \sum_{i=1}^n (H(x^{(i)}) - y^{(i)})^2$$

☞ Cost(Loss) Function

- 결국 Cost function의 값을 **최소**로 만드는 W 와 b 의 값을 찾아내는 것이 **학습**의 목표

$$cost(W, b) = \frac{1}{n} \sum_{i=1}^n (H(x^{(i)}) - y^{(i)})^2$$

MACHINE LEARNING WITH TENSORFLOW

☞ Tensors

- Types은 Tensor의 데이터 타입을 의미, Type 변경 가능

```
# Tensor의 type변경
import tensorflow as tf
import numpy as np

node1 = tf.constant([1,2,3], dtype=tf.int32)

node2 = tf.cast(node1,dtype=tf.float32)
# node2 = tf.cast(node1,dtype=np.float64)

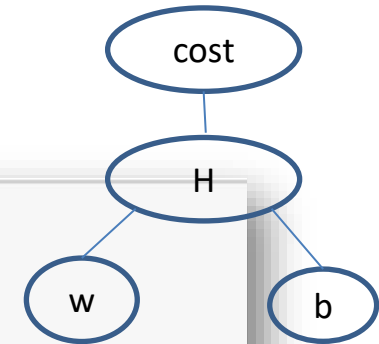
sess = tf.Session()

print(sess.run(node1))
print(sess.run(node2))
```

MACHINE LEARNING WITH TENSORFLOW

👉 Linear regression 실습 (1)

```
## Linear Regression 실습  
import tensorflow as tf  
  
# training data set  
x = [1,2,3]  
y = [1,2,3]  
  
# Weight & bias  
W = tf.Variable(tf.random_normal([1]), name="weight")  
b = tf.Variable(tf.random_normal([1]), name="bias")  
  
# Hypothesis  
H = W * x + b  
  
# Cost function  
cost = tf.reduce_mean(tf.square(H - y))
```



MACHINE LEARNING WITH TENSORFLOW

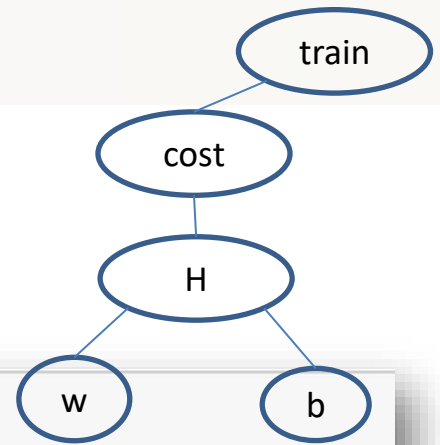
👉 Linear regression 실습 (2)

```
# cost function minimize
optimizer = tf.train.GradientDescentOptimizer(learning_rate=0.01)
train = optimizer.minimize(cost)

# runner 생성
sess = tf.Session()

# Variable 초기화
sess.run(tf.global_variables_initializer())

for step in range(3000):
    _, w_val, b_val, cost_val = sess.run([train, W, b, cost])
    if step % 300 == 0:
        print("W의 값 : {}, b의 값 : {}, cost의 값 : {}".format(w_val, b_val, cost_val))
```



MACHINE LEARNING WITH TENSORFLOW

👉 Linear regression 실습 - **placeholder**를 이용하여 초기데이터 세팅

```
## Linear Regression 실습
```

```
import tensorflow as tf
```

```
# training data set
```

```
x = tf.placeholder(dtype=tf.float32)
```

```
y = tf.placeholder(dtype=tf.float32)
```

```
x_data = [1,2,3]
```

```
y_data = [1,2,3]
```

```
#####
```

```
# 중간코드 생략
```

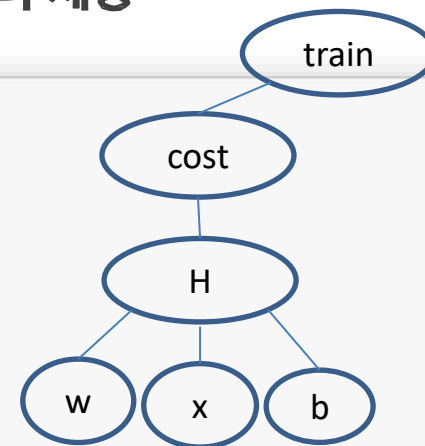
```
#####
```

```
for step in range(3000):
```

```
    _, w_val, b_val, cost_val = sess.run([train, W, b, cost],  
                                          feed_dict={x:x_data,y:y_data})
```

```
    if step % 300 == 0:
```

```
        print("W의 값 : {}, b의 값 : {}, cost의 값 : {}".format(w_val,b_val,cost_val))
```



MACHINE LEARNING WITH TENSORFLOW

☞ 학습 종료 후 데이터를 이용한 추정

```
#####  
# 상위코드 생략  
#####  
  
for step in range(3000):  
    _, w_val, b_val, cost_val = sess.run([train, W, b, cost],  
                                         feed_dict={x:x_data,y:y_data})  
  
    if step % 300 == 0:  
        print("W의 값 : {}, b의 값 : {}, cost의 값 : {}".format(w_val,b_val,cost_val))  
  
## prediction  
  
print(sess.run(H,feed_dict={x:[17]}))  
  
print(sess.run(H,feed_dict={x:[100,200,300]}))
```

MACHINE LEARNING WITH TENSORFLOW

☞ Cost(Loss) function 심화

- 목적은 Cost function을 최소화 시키는 W와 b의 값을 찾는 것.

```
# Weight & bias
W = tf.Variable(tf.random_normal([1]), name="weight")
b = tf.Variable(tf.random_normal([1]), name="bias")

# Hypothesis
H = W * x + b

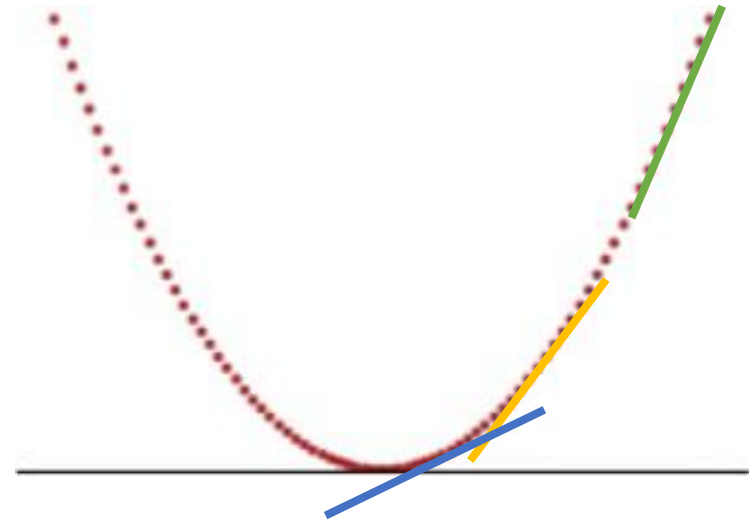
# Cost function
cost = tf.reduce_mean(tf.square(H - y))

# cost function minimize
optimizer = tf.train.GradientDescentOptimizer(learning_rate=0.01)
train = optimizer.minimize(cost)
```

MACHINE LEARNING WITH TENSORFLOW

☞ Cost function의 값을 최소화 시키는 W 를 어떻게 알아낼 수 있는가?

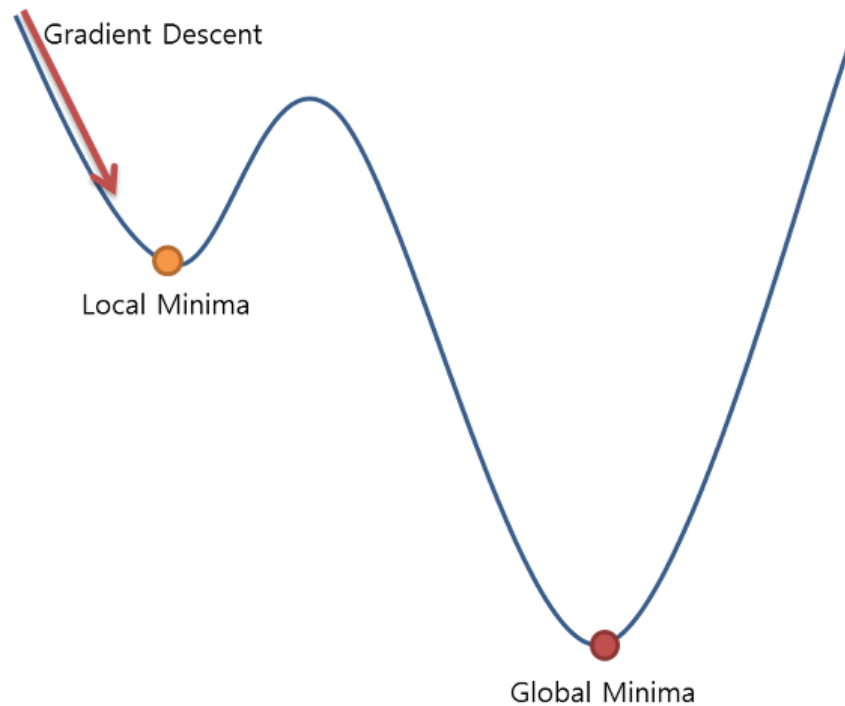
- Gradient Descent Algorithm을 이용
- 최소값을 찾기 위해 많이 사용되는 대표적인 Algorithm
- 미분을 이용



MACHINE LEARNING WITH TENSORFLOW

☞ Gradient Descent Algorithm의 예외 경사하강알고리즘

- Gradient Descent Algorithm은 항상 사용할 수 있는 것은 **아니다**.



MACHINE LEARNING WITH TENSORFLOW

☞ Multi-variable(multiple) linear regression

- 여러 개의 input을 이용한 linear regression

- 3번의 quiz 점수를 기반으로 시험성적 예측

x1(quiz1)	x2(quiz2)	x3(quiz3)	y(exam)
73	80	75	152
93	88	93	185
89	91	90	180
96	98	100	196
73	66	70	142

MACHINE LEARNING WITH TENSORFLOW

☞ Multi-variable(multiple) linear regression

- Hypothesis

- 변수가 1개일때의 Hypothesis

$$H(x) = Wx + b$$

- 변수가 3개일때의 Hypothesis

$$H(x_1, x_2, x_3) = w_1x_1 + w_2x_2 + w_3x_3 + b$$

MACHINE LEARNING WITH TENSORFLOW

☞ Multi-variable(multiple) linear regression

● Matrix를 이용한 Hypothesis

$$H(x_1, x_2, \dots, x_n) = \underline{w_1x_1 + w_2x_2 + \dots + w_nx_n} + b$$


$$(x_1 \quad x_2 \quad x_3) \cdot \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix} = (x_1w_1 + x_2w_2 + x_3w_3)$$


$$H(x) = XW + b$$

Matrix를 사용할 경우
관용적으로 대문자 사
용, X를 앞에 위치

MACHINE LEARNING WITH TENSORFLOW

☞ Multi-variable(multiple) linear regression

● 입력으로 여러 개의 행이 존재할 경우는 어떻게 하는가?

$$(x_1 \quad x_2 \quad x_3) \cdot \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix} = (x_1w_1 + x_2w_2 + x_3w_3)$$

x1(quiz1)	x2(quiz2)	x3(quiz3)	y(exam)
73	80	75	152
93	88	93	185
89	91	90	180
96	98	100	196
73	66	70	142

하나의 행
(instance)

MACHINE LEARNING WITH TENSORFLOW

☞ Multi-variable(multiple) linear regression

● 입력으로 여러 개의 행이 존재할 경우는 어떻게 하는가?

➢ Matrix의 곱셈을 활용

x1	x2	x3	y
73	80	75	152
93	88	93	185
89	91	90	180
96	98	100	196
73	66	70	142

$$\begin{pmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \\ x_{41} & x_{42} & x_{43} \\ x_{51} & x_{52} & x_{53} \end{pmatrix} \cdot \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix} = \begin{pmatrix} x_{11}w_1 + x_{12}w_2 + x_{13}w_3 \\ x_{21}w_1 + x_{22}w_2 + x_{23}w_3 \\ x_{31}w_1 + x_{32}w_2 + x_{33}w_3 \\ x_{41}w_1 + x_{42}w_2 + x_{43}w_3 \\ x_{51}w_1 + x_{52}w_2 + x_{53}w_3 \end{pmatrix}$$

$[5, 3] \quad [3, 1] \quad [5, 1]$

☞ Multi-variable(multiple) linear regression

- 기본이론

$$H(x) = Wx + b$$

- 실제구현

$$H(x) = XW + b$$

MACHINE LEARNING WITH TENSORFLOW

☞ Multi-variable(multiple) linear regression 실습

- 사용데이터

x1(quiz1)	x2(quiz2)	x3(quiz3)	y(exam)
73	80	75	152
93	88	93	185
89	91	90	180
96	98	100	196
73	66	70	142

MACHINE LEARNING WITH TENSORFLOW

☞ Multi-variable(multiple) linear regression 실습

```
# multiple linear regression 예제
import tensorflow as tf
import numpy as np
import pandas as pd

# training data set
x_data = [[73,80,75],
           [93,88,93],
           [89,91,90],
           [96,98,100],
           [73,66,70]]
y_data = [[152],[185],[180],[196],[142]]

# placeholder
X = tf.placeholder(shape=[None,3], dtype=tf.float32)
Y = tf.placeholder(shape=[None,1], dtype=tf.float32)

# Weight & bias
W = tf.Variable(tf.random_normal([3,1]), name="weight")
b = tf.Variable(tf.random_normal([1]), name="bias")
```


MACHINE LEARNING WITH TENSORFLOW

☞ Multi-variable(multiple) linear regression 실습

```
# Hypothesis
H = tf.matmul(X,W) + b

# Cost function
cost = tf.reduce_mean(tf.square(H-Y))

# train
train = tf.train.GradientDescentOptimizer(learning_rate=1e-5).minimize(cost)

# Session & 초기화
sess = tf.Session()
sess.run(tf.global_variables_initializer())

# 학습
for step in range(3000):
    _, cost_val = sess.run([train,cost], feed_dict={X:x_data, Y:y_data})
    if step % 300 == 0:
        print("cost : {}".format(cost_val))
```

MACHINE LEARNING WITH TENSORFLOW

☞ Multi-variable(multiple) linear regression 실습 – Ozone량 예측

- ozone량 예측
- CSV 파일로 부터 데이터를 loading후 학습과정을 진행하고 ozone량을 **예측하기** 위한 machine learning을 진행한다.
- 사용할 파일 : ozone.csv

MACHINE LEARNING WITH TENSORFLOW

☞ Multi-variable(multiple) linear regression 실습 – Ozone량 예측

```
# Ozone량 예측( multiple linear regression - tensorflow )
import tensorflow as tf
import numpy as np
import pandas as pd

# Data Loading
data = pd.read_csv("./data/ozone/ozone.csv", sep=",")
# display(data.head())
# 결측값 제거
data = data.dropna(how="any")
# display(data.head())
# 필요한 컬럼 추출
df = data[["Solar.R", "Wind", "Temp", "Ozone"]]
# Standardization, Normalization
df["Solar.R_Stan"] = (df["Solar.R"] - df["Solar.R"].mean()) / df["Solar.R"].std()
df["Wind_Stan"] = (df["Wind"] - df["Wind"].mean()) / df["Wind"].std()
df["Temp_Stan"] = (df["Temp"] - df["Temp"].mean()) / df["Temp"].std()
df["Ozone_Stan"] = (df["Ozone"] - df["Ozone"].mean()) / df["Ozone"].std()
display(df.head())
```

MACHINE LEARNING WITH TENSORFLOW

☞ Multi-variable(multiple) linear regression 실습 – Ozone량 예측

```
# training data set
x_data = df[["Solar_R_Stan", "Wind_Stan", "Temp_Stan"]].values
y_data = df["Ozone_Stan"].values.reshape(-1,1)

# placeholder
X = tf.placeholder(shape=[None,3], dtype=tf.float32)
Y = tf.placeholder(shape=[None,1], dtype=tf.float32)

# Weight & bias
W = tf.Variable(tf.random_normal([3,1]), name="weight")
b = tf.Variable(tf.random_normal([1]), name="bias")

# Hypothesis
H = tf.matmul(X,W) + b

# cost(Loss) function
cost = tf.reduce_mean(tf.square(H-Y))
```

MACHINE LEARNING WITH TENSORFLOW

☞ Multi-variable(multiple) linear regression 실습 – Ozone량 예측

```
# train
train = tf.train.GradientDescentOptimizer(learning_rate=0.1).minimize(cost)

# Session & 초기화
sess = tf.Session()
sess.run(tf.global_variables_initializer())

# 학습
for step in range(3000):
    _, cost_val = sess.run([train, cost], feed_dict={X:x_data, Y:y_data})
    if step % 300 == 0:
        print("cost : {}".format(cost_val))

# prediction
# Solar.R, Wind, Temp
input_data = [[190.0, 7.4, 67.0]] # shape을 맞추어서 입력
sess.run(H, feed_dict={X:input_data})
```


MACHINE LEARNING WITH TENSORFLOW

☞ Multi-variable(multiple) linear regression 실습 – Ozone량 예측(sklearn)

```
# Ozone량 예측( multiple linear regression - sklearn )
from sklearn import linear_model
import numpy as np
import pandas as pd

df = pd.read_csv("./data/ozone/ozone.csv", sep=",")
df = df.dropna(how="any")

# training data set
X = df[["Solar.R", "Wind", "Temp"]]
y = df["Ozone"]

lm = linear_model.LinearRegression()
lm.fit(X,y)
print("W의 값 : {}".format(lm.coef_))
print("intercept : {}".format(lm.intercept_))

predictions = lm.predict([[190.0, 7.4, 67.0]])
print("예측값 : {}".format(predictions))
```

☞ Logistic regression

- Binary Classification

- **분류 알고리즘 중 정확도가 높은 알고리즘**
- Email 판별 : Spam인지 아닌지 판별 (Spam이면 1, 아니면 0)
- 주식의 주가가 오를지 **떨어질지** 판별 (오르면 1, 아니면 0)
- 신용카드 사용 시 **도난카드인지** 아닌지 판별 (도난카드일 경우 1, 정상사용일 경우 0)

- Machine Learning이 0과 1사이의 값으로 예측.

- 통상적으로 0.5 이상이면 1로 간주, 0.5 미만이면 0으로 간주

MACHINE LEARNING WITH TENSORFLOW

☞ Logistic regression 예

- 시험공부 시간을 기준으로 한 Pass, Fail

x(시간)	y(결과)
10	PASS(1)
8	PASS(1)
5	FAIL(0)
2	FAIL(0)
1	FAIL(0)

MACHINE LEARNING WITH TENSORFLOW

☞ Linear regression으로는 해결할 수 없는가?

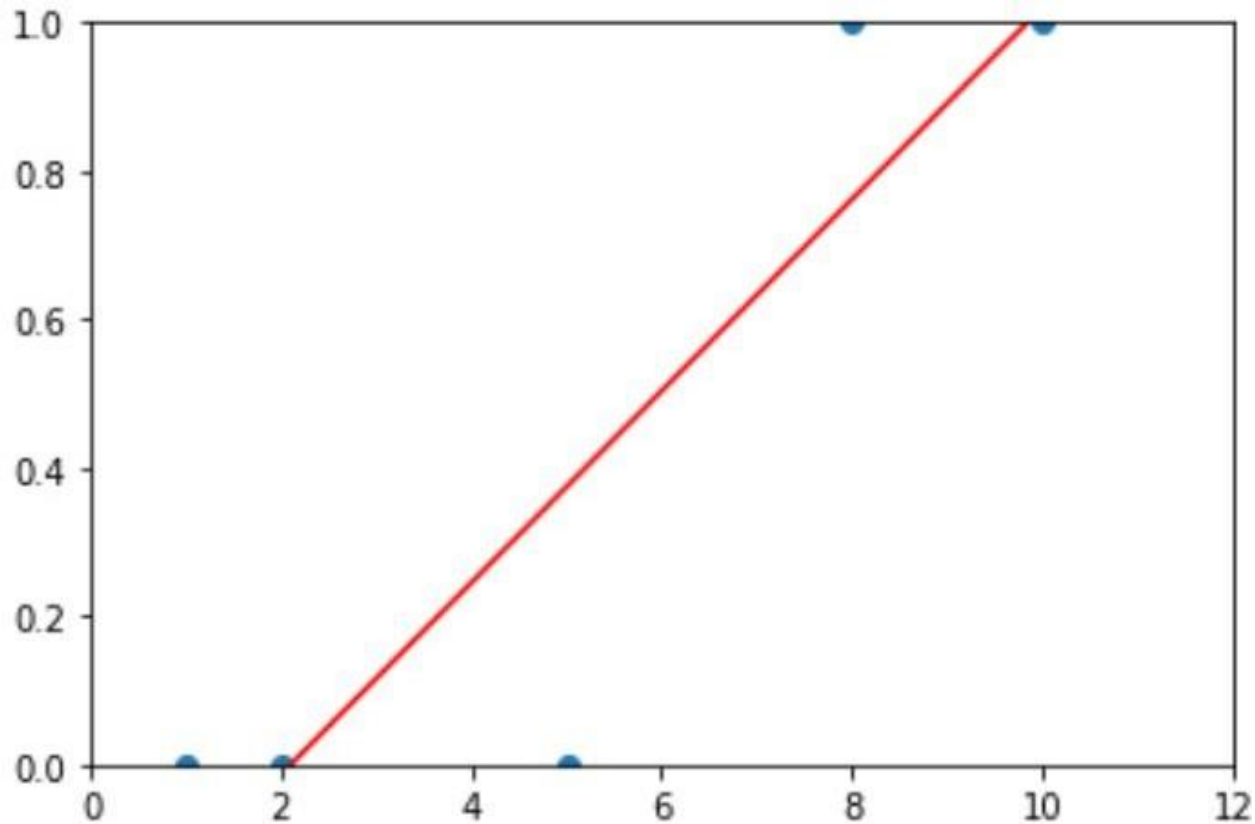
```
%matplotlib inline
### Logistic Regression이 왜 필요한가?
### 0과 1값으로 표현된 데이터를 Linear Regression으로 해결할 수 없는가?
### scipy를 이용해서 알아보자.
from scipy import stats
import numpy as np
import matplotlib
import matplotlib.pyplot as plt

x = np.array([1,2,5,8,10])
y = np.array([0,0,0,1,1])

result = stats.linregress(x,y)
print(result)
plt.scatter(x,y)
plt.plot(x,x*result[0]+result[1],"r")
plt.ylim([0,1])
plt.xlim([0,12])
plt.show()
print((0.5 - result.intercept) / result.slope)
```

MACHINE LEARNING WITH TENSORFLOW

👉 Linear regression으로는 해결할 수 없는가?



5.973684210526316

MACHINE LEARNING WITH TENSORFLOW

☞ Linear regression으로는 해결할 수 없는가?

- 데이터에 따라 정확하지 않은 값을 도출할 수 있다.

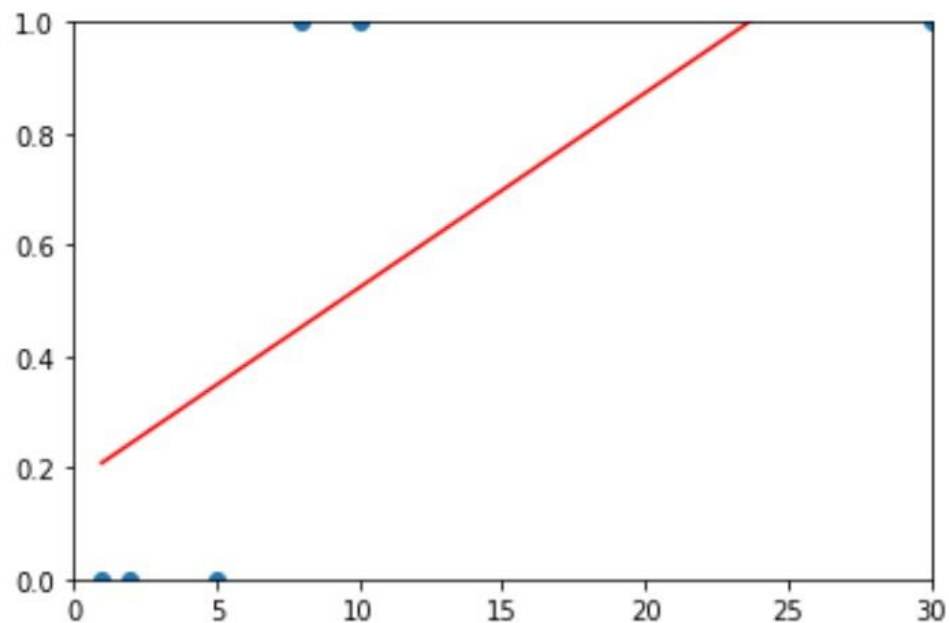
x(시간)	y(결과)
30	PASS(1)
10	PASS(1)
8	PASS(1)
5	FAIL(0)
2	FAIL(0)
1	FAIL(0)

데이터추가

MACHINE LEARNING WITH TENSORFLOW

👉 Linear regression으로는 해결할 수 없는가?

- 데이터에 따라 그래프가 변한다.
- 데이터에 따라 1보다 크거나 0보다 작은 값이 도출될 수 있다.



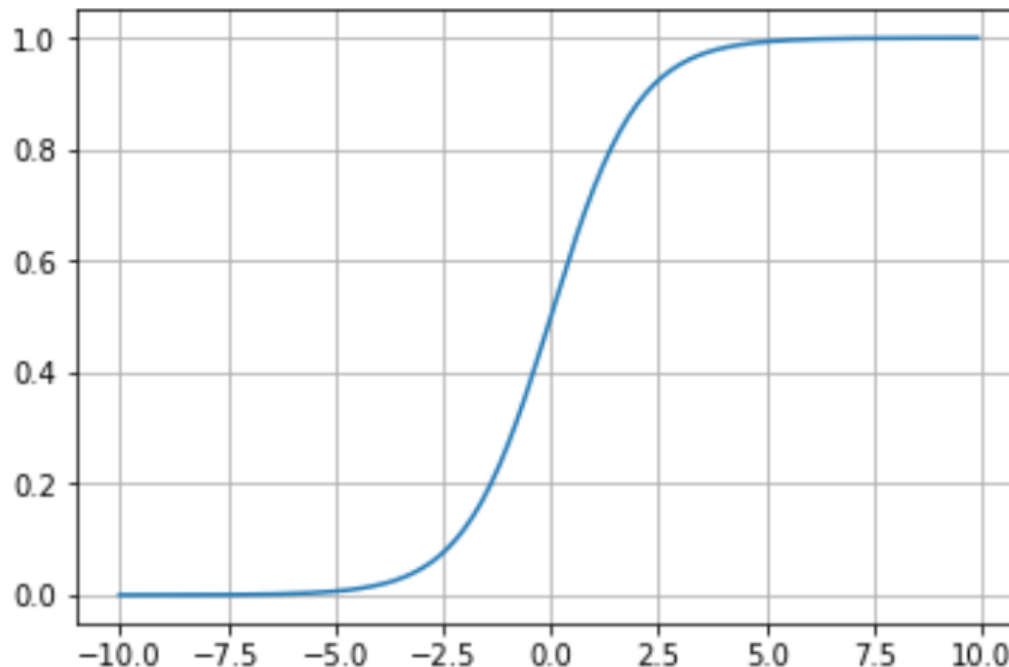
9.333333333333334

MACHINE LEARNING WITH TENSORFLOW

☞ Sigmoid function 응용

- H의 값이 0과 1사이의 값으로 **바인딩** 되어야 한다.
 - sigmoid function

$$f(x) = \frac{1}{1 + e^{-(x)}}$$



☞ Sigmoid function 응용

- sigmoid function은 **입력으로 어떤 값이 들어가도 0과 1사이의 값으로 바인딩 된다.**
 - 기존의 linear regression의 hypothesis를 입력으로 Sigmoid function에 인자로 사용.
 - 따라서 Logistic regression의 Hypothesis는 **다음처럼 표현된다.**

$$H(x) = \frac{1}{1 + e^{-(XW+b)}}$$

MACHINE LEARNING WITH TENSORFLOW

☞ Cost function(Linear regression)

$$H(x) = XW + b$$

$$cost(W, b) = \frac{1}{n} \sum_{i=1}^n (H(x^{(i)}) - y^{(i)})^2$$

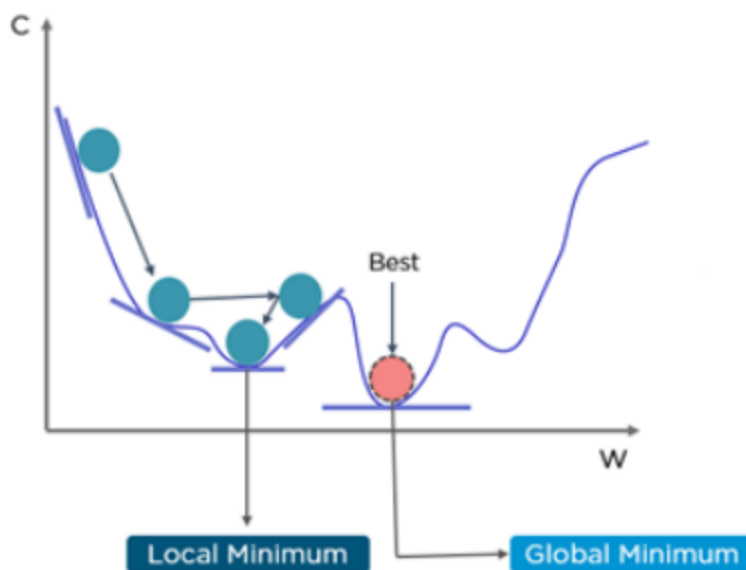


MACHINE LEARNING WITH TENSORFLOW

☞ Cost function(Logistic regression)

$$H(x) = \frac{1}{1 + e^{-(xW+b)}}$$

$$cost(W, b) = \frac{1}{n} \sum_{i=1}^n (H(x^{(i)}) - y^{(i)})^2$$



MACHINE LEARNING WITH TENSORFLOW

☞ New Cost function(Logistic regression)

- local minimum을 해결하기 위해 log를 이용

$$cost(H(x), y) = -y \log(H(x)) - (1 - y) \log(1 - H(x))$$

- Minimize Cost

➤ cost를 최소화하기 위해 Gradient Descent algorithm을 이용

MACHINE LEARNING WITH TENSORFLOW

👉 실습

● 사용할 데이터

➤ 영어시험

x1(시간)	x2(체류기간)	y(결과)
10	0	PASS(1)
8	1	PASS(1)
3	3	PASS(1)
2	3	PASS(1)
5	1	FAIL(0)
2	0	FAIL(0)
1	0	FAIL(0)

MACHINE LEARNING WITH TENSORFLOW

👉 실습

```
## Logistic regression 실습예제
import tensorflow as tf

# training data set
x_data = [[1,0],
           [2,0],
           [5,1],
           [2,3],
           [3,3],
           [8,1],
           [10,0]]
y_data = [[0],[0],[0],[0],[1],[1],[1]]

# placeholder
X = tf.placeholder(shape=[None,2], dtype=tf.float32)
Y = tf.placeholder(shape=[None,1], dtype=tf.float32)

# Weight & bias
W = tf.Variable(tf.random_normal([2,1]), name="weight")
b = tf.Variable(tf.random_normal([1]), name="bias")
```

MACHINE LEARNING WITH TENSORFLOW

👉 실습

```
# Hypothesis
logits = tf.matmul(X,W) + b
H = tf.sigmoid(logits)

# Cost function
# cost = -tf.reduce_mean(Y * tf.log(H) + (1-Y) * tf.log(1-H))
cost = tf.reduce_mean(tf.nn.sigmoid_cross_entropy_with_logits(logits=logits, labels=Y))

# train
train = tf.train.GradientDescentOptimizer(learning_rate=0.1).minimize(cost)

# session & 초기화
sess = tf.Session()
sess.run(tf.global_variables_initializer())

# 학습
for step in range(3000):
    _, cost_val = sess.run([train,cost], feed_dict={X:x_data, Y:y_data})
    if step % 300 == 0 :
        print("cost : {}".format(cost_val))
```

MACHINE LEARNING WITH TENSORFLOW

👉 실습

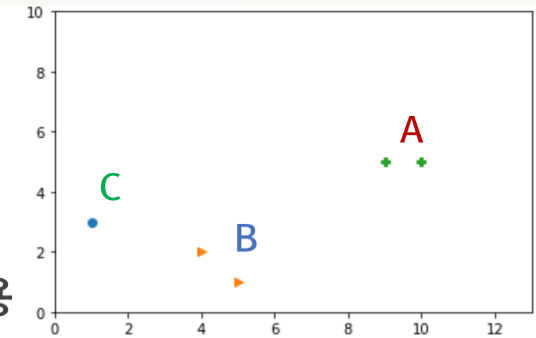
```
# Accuracy
predict = tf.cast( H > 0.5, dtype=tf.float32 )
accuracy = tf.reduce_mean(tf.cast(tf.equal(predict,Y), dtype=tf.float32))

print("정확도 : {}".format(sess.run(accuracy, feed_dict={X:x_data, Y:y_data})))
print("예측 : {}".format(sess.run(H, feed_dict={X:[[5,2]]})))
```

MACHINE LEARNING WITH TENSORFLOW

👉 Multinomial Classification

- Logistic classification의 개념을 Multinomial classification에 적용

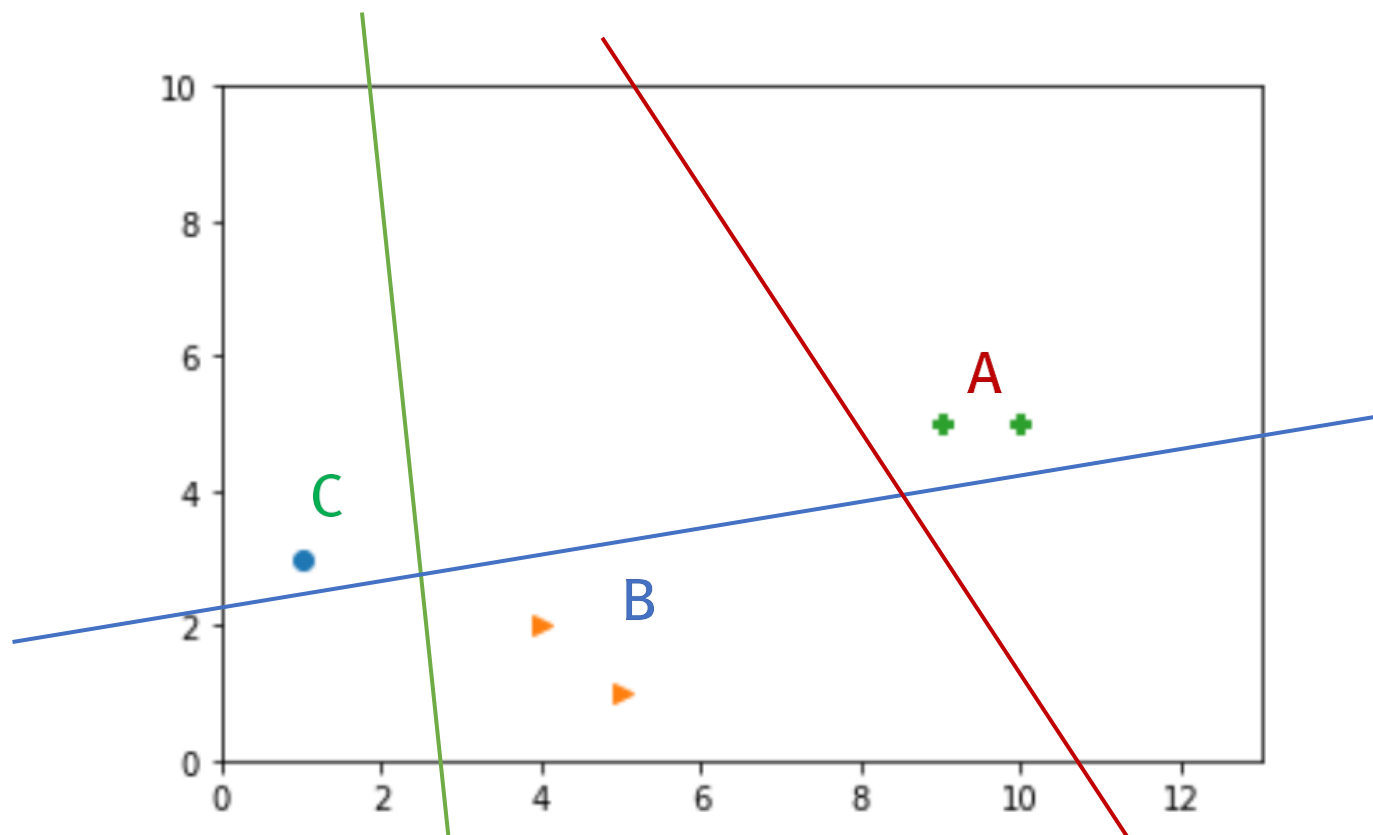


x1(Quiz1)	x2(Quiz2)	x3(Quiz3)	x4(출석)	y(결과)
10	7	8	5	A
8	8	9	4	A
7	8	2	3	B
6	3	9	3	B
7	5	7	4	B
3	5	6	2	C
2	4	3	1	C

MACHINE LEARNING WITH TENSORFLOW

☞ Multinomial Classification

- 3개의 Binary classification을 이용



MACHINE LEARNING WITH TENSORFLOW

☞ Multinomial Classification

- 3개의 Binary classification을 이용 – Probability

$$(x_1 \quad x_2 \quad x_3) \cdot \begin{pmatrix} w_{A1} & w_{B1} & w_{C1} \\ w_{A2} & w_{B2} & w_{C2} \\ w_{A3} & w_{B3} & w_{C3} \end{pmatrix}$$



$$(x_1 w_{A1} + x_2 w_{A2} + x_3 w_{A3} \quad x_1 w_{B1} + x_2 w_{B2} + x_3 w_{B3} \quad x_1 w_{C1} + x_2 w_{C2} + x_3 w_{C3})$$



probability

(0.4 0.5 0.1)

A

B

C

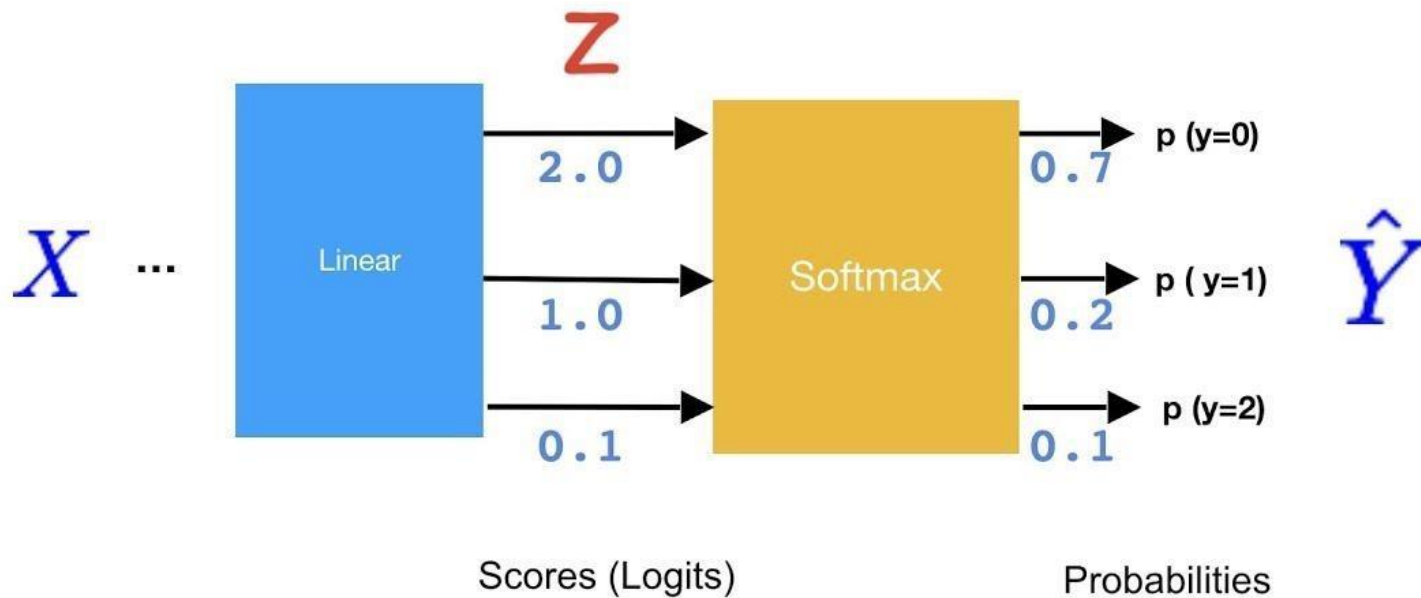
0과 1사이의 값이고
모두 더하면 1

MACHINE LEARNING WITH TENSORFLOW

👉 Multinomial Classification – Softmax

Meet Softmax

$$\sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}} \quad \text{for } j = 1, \dots, K.$$



MACHINE LEARNING WITH TENSORFLOW

☞ Multinomial Classification – ONE-HOT Encoding

y(결과)		
A		
A		
B		
B		
B		
C		
C		



A	B	C
1	0	0
1	0	0
0	1	0
0	1	0
0	1	0
0	0	1
0	0	1

MACHINE LEARNING WITH TENSORFLOW

👉 Multinomial Classification – ONE-HOT Encoding

		A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
L	→	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
U	→	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
K	→	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
A	→	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
S	→	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0

👉 Multinomial Classification

Hypothesis

```
logits = tf.matmul(X,W) + b
```

```
H = tf.nn.softmax(tf.matmul(X,W) + b)
```

Cost function

```
cost = tf.reduce_mean(  
tf.nn.softmax_cross_entropy_with_logits_v2(logits=logits,  
labels=y_data))
```

MACHINE LEARNING WITH TENSORFLOW

👉 실습

● 사용할 데이터

➤ 성적처리

x1(Quiz1)	x2(Quiz2)	x3(Quiz3)	x4(출석)	y(결과)
10	7	8	5	A
8	8	9	4	A
7	8	2	3	B
6	3	9	3	B
7	5	7	4	B
3	5	6	2	C
2	4	3	1	C

MACHINE LEARNING WITH TENSORFLOW

👉 실습

```
## multinomial classification 실습예제
```

```
import tensorflow as tf
```

```
# training data set
```

```
x_data = [[10,7,8,5],  
          [8,8,9,4],  
          [7,8,2,3],  
          [6,3,9,3],  
          [7,5,7,4],  
          [3,5,6,2],  
          [2,4,3,1]]
```

```
y_data = [[1,0,0],  
          [1,0,0],  
          [0,1,0],  
          [0,1,0],  
          [0,1,0],  
          [0,0,1],  
          [0,0,1]]
```

MACHINE LEARNING WITH TENSORFLOW

👉 실습

```
# placeholder
X = tf.placeholder(shape=[None,4], dtype=tf.float32)
Y = tf.placeholder(shape=[None,3], dtype=tf.float32)

# Weight & bias
W = tf.Variable(tf.random_normal([4,3]), name="weight")
b = tf.Variable(tf.random_normal([3]), name="bias")

# Hypothesis
logits = tf.matmul(X,W) + b
H = tf.nn.softmax(logits)

# cost function
cost = tf.reduce_mean(tf.nn.softmax_cross_entropy_with_logits_v2(logits=logits, labels=Y))

# train
train = tf.train.GradientDescentOptimizer(learning_rate=0.01).minimize(cost)
```

MACHINE LEARNING WITH TENSORFLOW

👉 실습

```
# session & 초기화
sess = tf.Session()
sess.run(tf.global_variables_initializer())

# 학습
for step in range(3000):
    _, cost_val = sess.run([train, cost], feed_dict={X:x_data, Y:y_data})
    if step % 300 == 0:
        print("Cost : {}".format(cost_val))

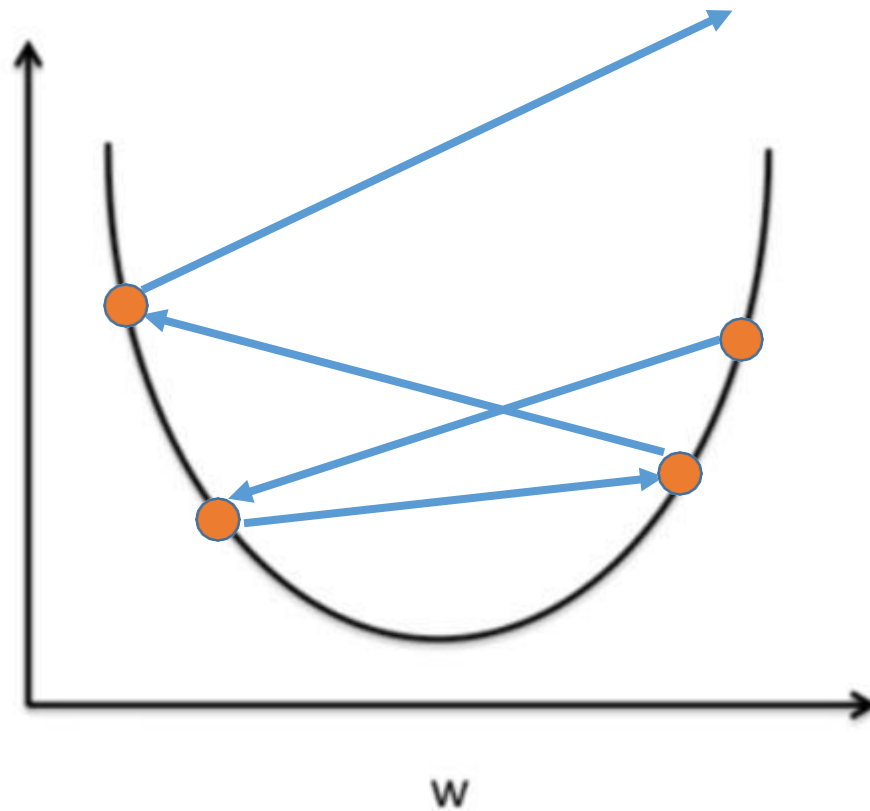
# accuracy
predict = tf.argmax(H, 1) # 두번째 인자는 one-hot encoding을 적용할 차원
correct = tf.equal(predict, tf.argmax(Y, 1))
accuracy = tf.reduce_mean(tf.cast(correct, dtype=tf.float32))

print("정확도 : {}".format(sess.run(accuracy, feed_dict={X:x_data, Y:y_data})))
print(sess.run(H, feed_dict={X:[[9,4,3,6]]}))
print(sess.run(predict, feed_dict={X:[[9,4,3,6]]}))
```


MACHINE LEARNING WITH TENSORFLOW

👉 Large learning rate

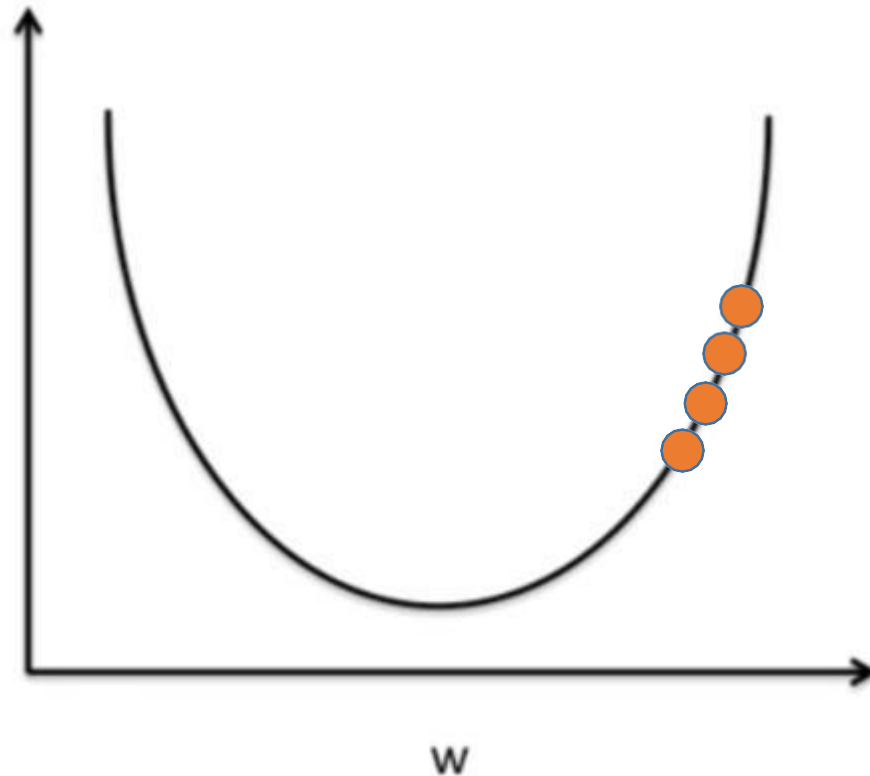
- Overshooting



MACHINE LEARNING WITH TENSORFLOW

☞ Small learning rate

- local minimum 문제에 봉착할 수 있음.



MACHINE LEARNING WITH TENSORFLOW

☞ learning rate 값 결정

- 여러 개의 값으로 Test
- Cost function의 값을 **변화를 살펴보면서** 적절한 learning rate를 **찾아야** 한다.
- **데이터와 실행** 환경에 따라 learning rate는 **다르기 때문에** 경험적으로 찾아야 함.
- 초기값은 0.01 **정도로 세팅한** 후 값을 수정해 가면서 **적절한 값을 찾는다.**

MACHINE LEARNING WITH TENSORFLOW

👉 입력데이터의 Preprocessing (Standardization)

The diagram illustrates the standardization formula with callouts for each variable:

- x : X
- μ_j : X의 평균
- σ_j : X의 분산
- x'_j : new_X

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}$$

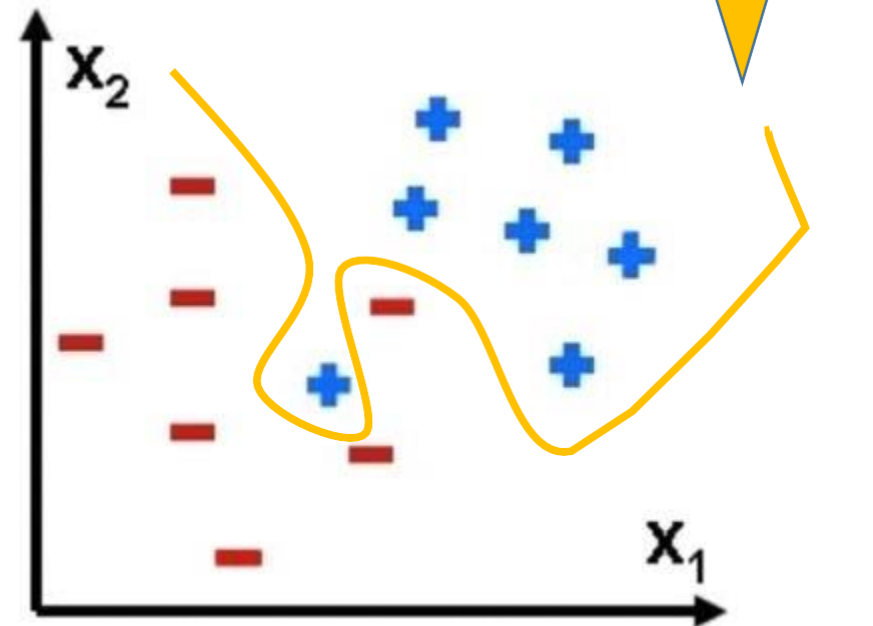
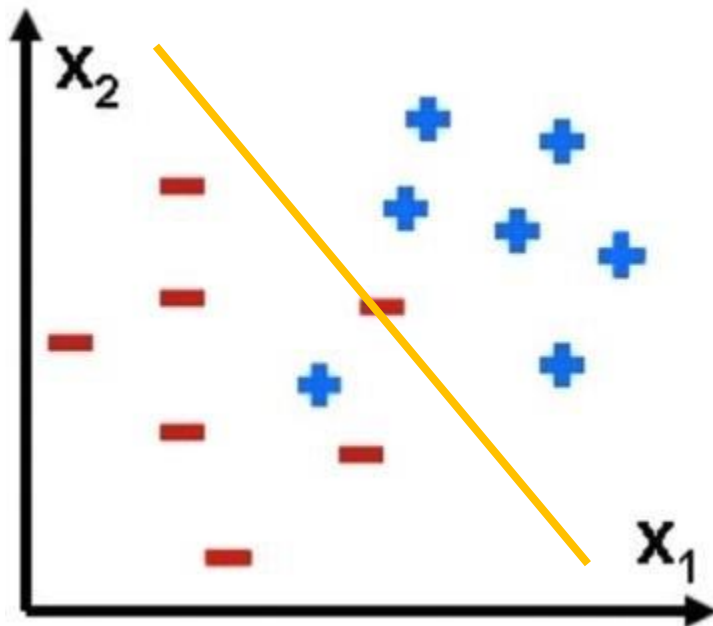
👉 입력데이터의 Preprocessing (Normalization – min max scale)

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

MACHINE LEARNING WITH TENSORFLOW

👉 Overfitting이란?

- Training data set에는 아주 적합한 모델이지만 실제 데이터에는 적용이 잘 안되는 경우
 - training data set에 최적화된 모델



☞ Overfitting의 해결방안

- 많은 training data를 이용
- feature의 개수를 줄인다. (중복되거나 필요치 않은 feature를 생략)
- Regularization
 - weight값이 클 수록 그래프가 커브를 그리게 되고 overfitting이 일어나게 된다.
 - weight값이 너무 큰 값이 되지 않도록 조절.

DEEP LEARNING WITH TENSORFLOW

☞ Artificial Intelligence

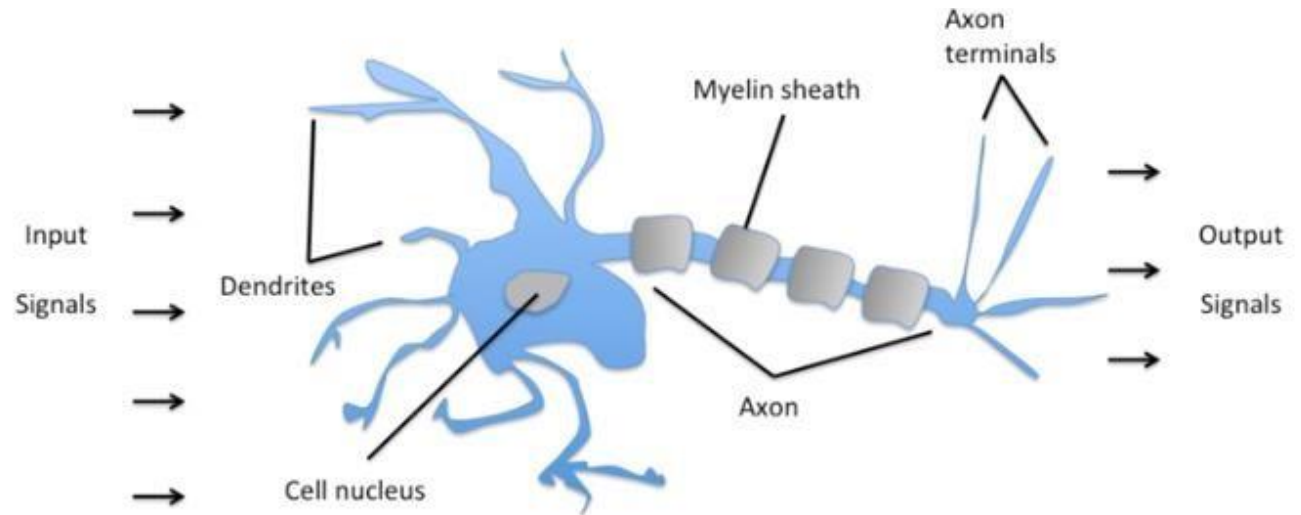
- 궁극의 목표는 인공지능을 구현해내는 것



DEEP LEARNING WITH TENSORFLOW

👉 인간의 뇌를 연구

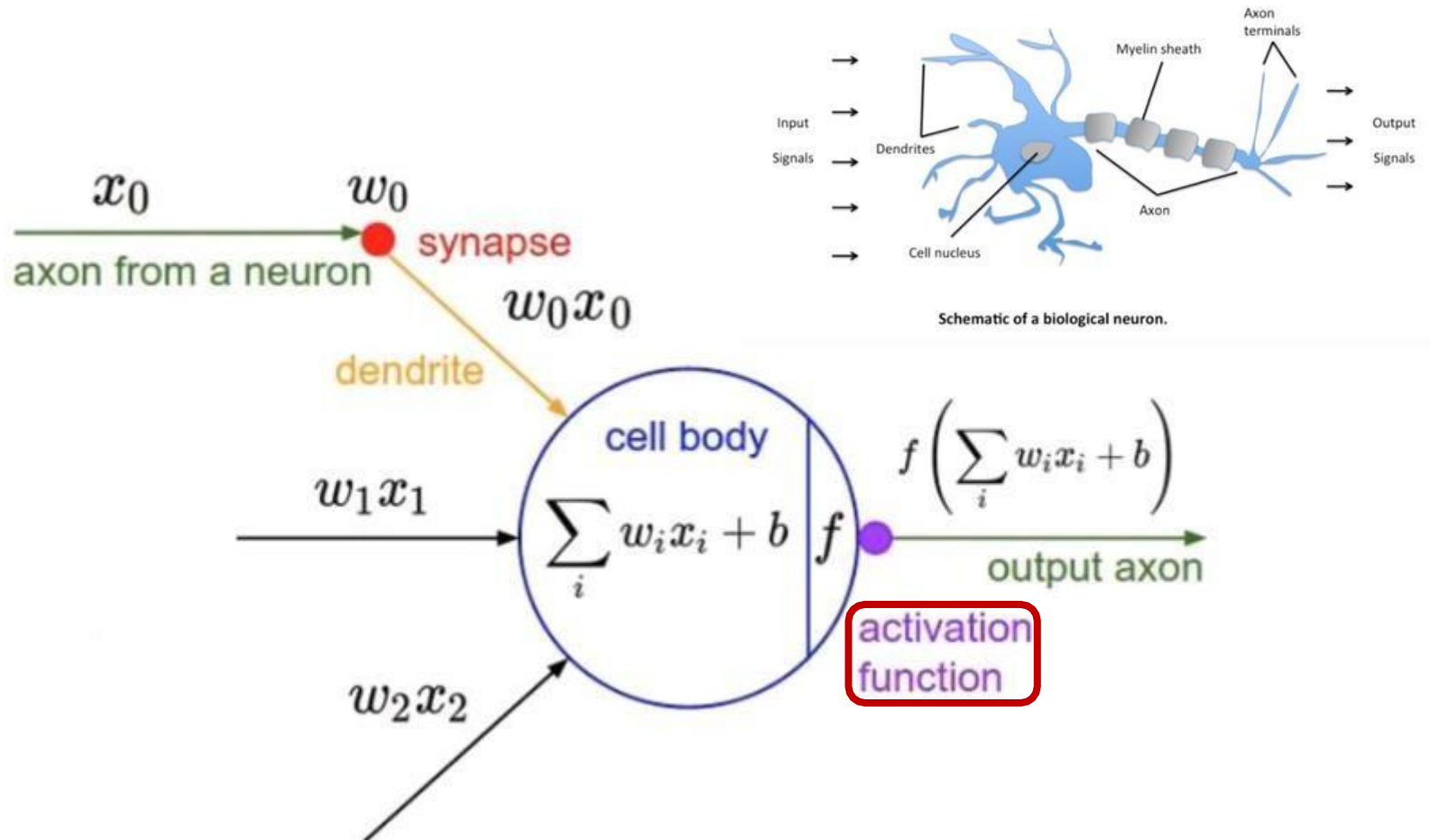
- 굉장히 복잡하게 연결되어 있지만 의외로 단순하게 동작



Schematic of a biological neuron.

DEEP LEARNING WITH TENSORFLOW

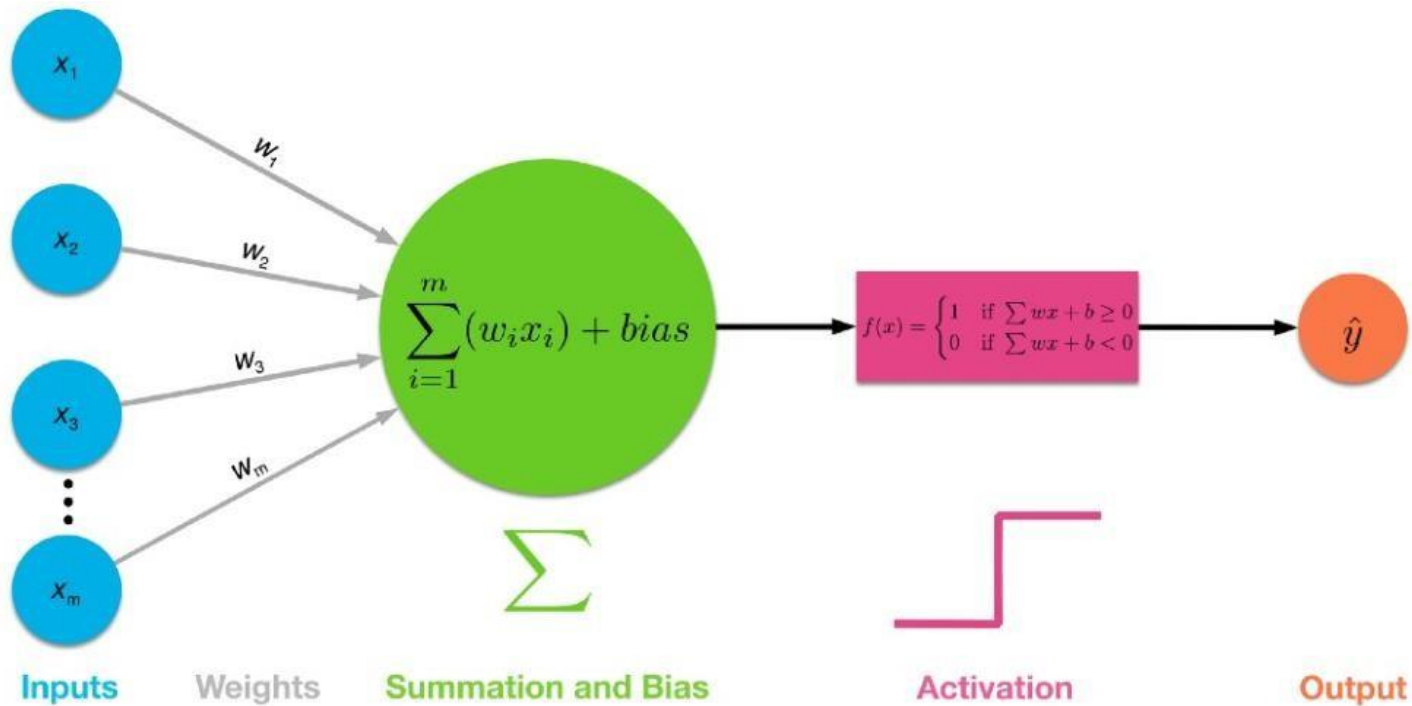
☞ Activation Function



DEEP LEARNING WITH TENSORFLOW

☞ Single-layer Perceptron Network

- Perceptron : 일종의 학습 기계로서 뇌의 학습 기능을 모델링한 기계.



DEEP LEARNING WITH TENSORFLOW

☞ AND/OR Problem

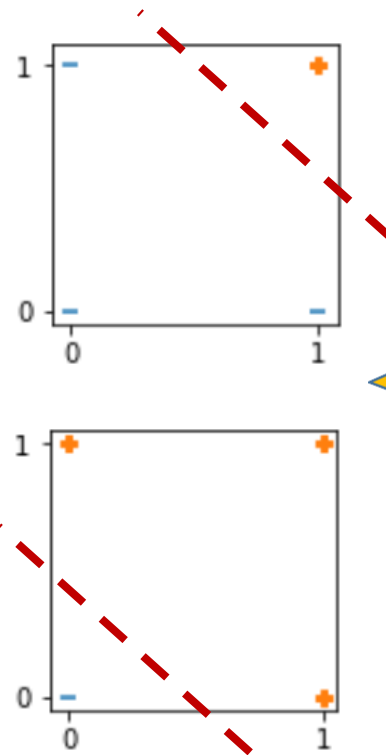
- AND/OR 진리표를 프로그램으로 학습한 후 예측할 수 있다면 이를 조합해서 인공지능 기계를 만들 수 있을 것이라 생각

AND Truth Table

Inputs		Output
A	B	$Y = A.B$
0	0	0
0	1	0
1	0	0
1	1	1

OR Truth Table

Inputs		Output
A	B	$Y = A+B$
0	0	0
0	1	1
1	0	1
1	1	1



가능!!

DEEP LEARNING WITH TENSORFLOW

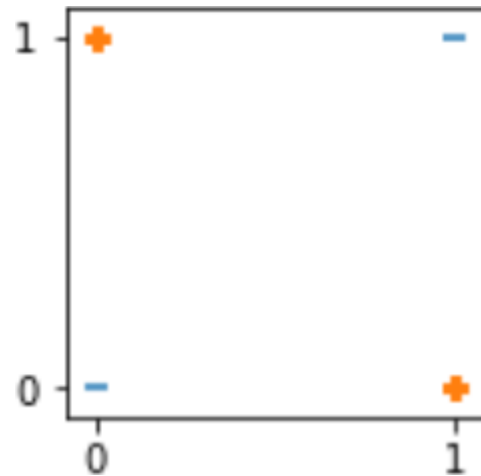
☞ XOR Problem

● XOR문제를 풀 수 있을까?

➤ 구현이 안됨 (선을 그을 수 없다)

XOR Truth Table

Inputs		Output
A	B	$Y = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0



DEEP LEARNING WITH TENSORFLOW

☞ XOR Problem with Logistic Regression

```
### XOR Problem with logistic regression
import tensorflow as tf

# training data set
x_data = [ [0,0], [0,1], [1,0], [1,1] ]
y_data = [ [0], [1], [1], [0] ]

# placeholder
X = tf.placeholder(shape=[None,2], dtype=tf.float32)
Y = tf.placeholder(shape=[None,1], dtype=tf.float32)

# Weight & bias
W = tf.Variable(tf.random_normal([2,1]), name="weight")
b = tf.Variable(tf.random_normal([1]), name="bias")

# Hypothesis
logits = tf.matmul(X,W) + b
H = tf.sigmoid(logits)

# cost
cost = tf.reduce_mean(tf.nn.sigmoid_cross_entropy_with_logits(logits=logits, labels=Y))
```

DEEP LEARNING WITH TENSORFLOW

☞ XOR Problem with Logistic Regression

```
# train
train = tf.train.GradientDescentOptimizer(learning_rate=0.01).minimize(cost)

# Session & 초기화
sess = tf.Session()
sess.run(tf.global_variables_initializer())

# 학습
for step in range(30000):
    _, cost_val = sess.run([train, cost], feed_dict={X:x_data, Y:y_data})
    if step % 3000 == 0:
        print("Cost : {}".format(cost_val))

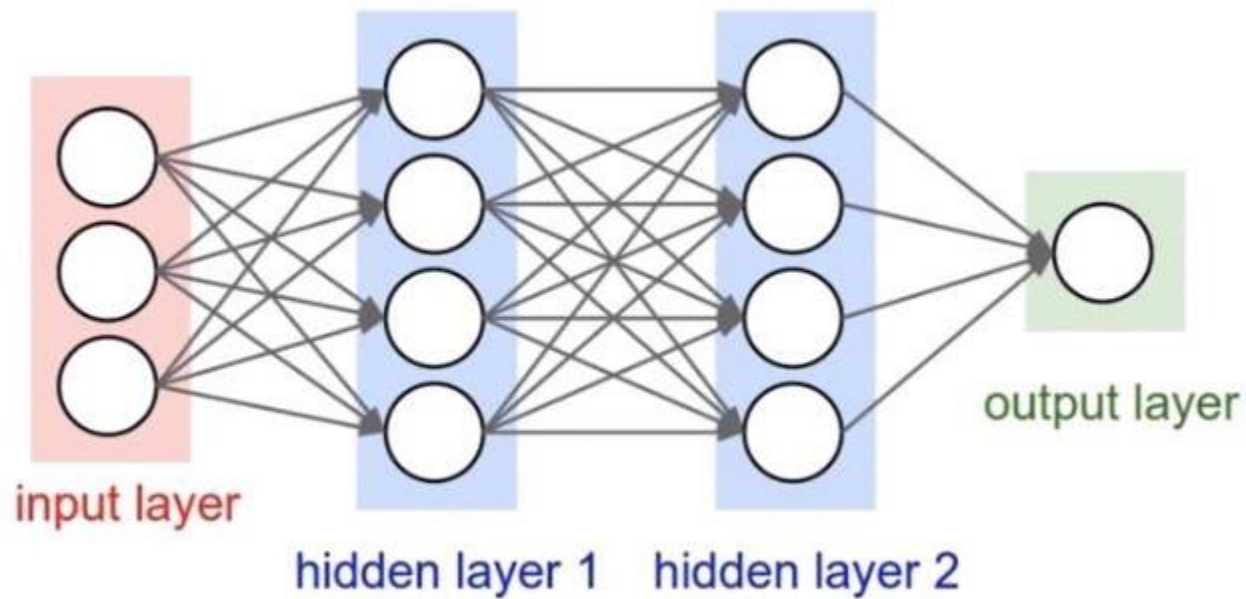
# accuracy
predict = tf.cast( H > 0.5, dtype=tf.float32)
correct = tf.equal(predict, Y)
accuracy = tf.reduce_mean(tf.cast(correct, dtype=tf.float32))

print("accuracy : {}".format(sess.run(accuracy, feed_dict={X:x_data, Y:y_data})))
```


DEEP LEARNING WITH TENSORFLOW

👉 MLP의 문제점

- No one on earth had found a viable way to train



DEEP LEARNING WITH TENSORFLOW

☞ XOR Problem with Neural Network

```
### XOR Problem with Neural Network
import tensorflow as tf

# training data set
x_data = [ [0,0], [0,1], [1,0], [1,1] ]
y_data = [ [0], [1], [1], [0] ]

# placeholder
X = tf.placeholder(shape=[None,2], dtype=tf.float32)
Y = tf.placeholder(shape=[None,1], dtype=tf.float32)

# Weight & bias
W1 = tf.Variable(tf.random_normal([2,2]), name="weight1")
b1 = tf.Variable(tf.random_normal([2]), name="bias1")
layer1 = tf.sigmoid(tf.matmul(X,W1) + b1)

W2 = tf.Variable(tf.random_normal([2,1]), name="weight2")
b2 = tf.Variable(tf.random_normal([1]), name="bias2")
```

DEEP LEARNING WITH TENSORFLOW

☞ XOR Problem with Neural Network

```
# Hypothesis
logits = tf.matmul(layer1,W2) + b2
H = tf.sigmoid(logits)

# cost
cost = tf.reduce_mean(tf.nn.sigmoid_cross_entropy_with_logits(logits=logits, labels=Y))

# train
train = tf.train.GradientDescentOptimizer(learning_rate=0.1).minimize(cost)

# Session & 초기화
sess = tf.Session()
sess.run(tf.global_variables_initializer())

# 학습
for step in range(30000):
    _, cost_val = sess.run([train,cost], feed_dict={X:x_data, Y:y_data})
    if step % 3000 == 0:
        print("cost : {}".format(cost_val))
```

DEEP LEARNING WITH TENSORFLOW

☞ Deep & Wide XOR Problem with Neural Network

- 이전 Neural Network을 이용한 XOR문제를 더 잘 해결하기 위해 layer를 추가

```
# Weight & bias
W1 = tf.Variable(tf.random_normal([2,10]), name="weight1")
b1 = tf.Variable(tf.random_normal([10]), name="bias1")
layer1 = tf.sigmoid(tf.matmul(X,W1) + b1)

W2 = tf.Variable(tf.random_normal([10,20]), name="weight2")
b2 = tf.Variable(tf.random_normal([20]), name="bias2")
layer2 = tf.sigmoid(tf.matmul(layer1,W2) + b2)

W3 = tf.Variable(tf.random_normal([20,10]), name="weight3")
b3 = tf.Variable(tf.random_normal([10]), name="bias3")
layer3 = tf.sigmoid(tf.matmul(layer2,W3) + b3)

W4 = tf.Variable(tf.random_normal([10,1]), name="weight4")
b4 = tf.Variable(tf.random_normal([1]), name="bias4")

# Hypothesis
logits = tf.matmul(layer3,W4) + b4
H = tf.sigmoid(logits)
```

DEEP LEARNING WITH TENSORFLOW

☞ Sigmoid vs. ReLU (Rectified Linear Unit)

- Neural Network에서는 더 이상 sigmoid를 사용하지 않는다.

```
layer1 = tf.sigmoid(tf.matmul(X,W1) + b1)
```



```
layer1 = tf.nn.relu(tf.matmul(X,W1) + b1)
```